

Отчет по лабораторной работе №12

Операционные системы

Зарипова Мария Вячеславовна

Содержание

1	Цель работы	1
2	Задание	1
3	Теоретическое введение	2
4	Выполнение лабораторной работы	2
5	Выводы.....	7
6	Ответы на контрольные вопросы	8

1 Цель работы

Цель данной лабораторной работы — изучить основы программирования в оболочке ОС UNIX, научиться писать командные файлы для автоматизации задач резервного копирования, обработки аргументов командной строки, создания аналогов системных команд и подсчёта файлов по расширениям.

2 Задание

Резервная копия самого скрипта

1. Написать скрипт, который при запуске делает резервную копию самого себя (файла с исходным кодом) в директорию `~/backup`. Файл должен архивироваться архиватором `tar`.
2. Обработка произвольного числа аргументов (>10)
Написать командный файл, обрабатывающий любое произвольное число аргументов командной строки, в том числе превышающее десять. Скрипт должен распечатывать значения всех переданных аргументов.
3. Аналог команды `ls`
Написать командный файл — аналог команды `ls` (без использования самой команды `ls` и `dir`). Скрипт должен выдавать информацию о содержимом каталога и выводить права доступа к каждому файлу.
4. Подсчёт файлов по расширению
Написать командный файл, который получает в качестве аргументов формат файла (например `.txt`) и путь к директории, после чего вычисляет количество таких файлов в указанной директории.

3 Теоретическое введение

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек:

- Оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций;
- С-оболочка (или csh) — надстройка над оболочкой Борна, использующая С-подобный синтаксис команд с возможностью сохранения истории выполнения команд;
- оболочка Корна (или ksh) — напоминает оболочку С, но операторы управления программой совместимы с операторами оболочки Борна;
- BASH (Bourne Again Shell) — в основе своей совмещает свойства оболочек С и Корна (разработка компании Free Software Foundation).
- POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ.

В данной лабораторной работе используются переменные, подстановки, управляющие конструкции (if, for), а также работа с файлами и правами доступа.

4 Выполнение лабораторной работы

Создаю каталог для лабораторной работы и командный файл для первого задания (рис. 1).

```
mvzaripova@fedora:~$ mkdir -p lab_shell
mvzaripova@fedora:~$ cd lab_shell

mvzaripova@fedora:~/lab_shell$ chmod +x backup_self.sh
mvzaripova@fedora:~/lab_shell$ ./backup_self.sh
Резервная копия ./backup_self.sh создана в ~/backup/backup_self.tar.gz
```

Figure 1: Создание каталога lab_shell и переход в него

Листинг командного файла представлен на (рис. 2).

```
#!/bin/bash
mkdir -p ~/backup
tar -czf ~/backup/backup_self.tar.gz "$0"
echo "Резервная копия $0 создана в ~/backup/backup_self.tar.gz"
```

Figure 2: Код программы

```
#!/bin/bash
```

```
mkdir -p ~/backup
```

```
tar -czf ~/backup/backup_self.tar.gz "$0"
```

```
echo "Резервная копия $0 создана в ~/backup/backup_self.tar.gz"
```

Делаю скрипт исполняемым и запускаю его (рис. 3).

```
mvzaripova@fedora:~/lab_she11$ chmod +x backup_self.sh
mvzaripova@fedora:~/lab_she11$ ./backup_self.sh
Резервная копия ./backup_self.sh создана в ~/backup/backup_self.tar.gz
mvzaripova@fedora:~/lab_she11$ ls -l ~/backup
итого 4
-rw-r--r--. 1 mvzaripova mvzaripova 248 апр 28 22:15 backup_self.tar.gz
```

Figure 3: Запуск backup_self.sh

Проверка создания архива в каталоге ~/backup (рис. 4).

```
mvzaripova@fedora:~
GNU nano 8.5
#!/bin/bash
echo "Всего аргументов: $#"
```

```
echo "Имя скрипта: $0"
```

```
echo "Все аргументы (\$*): $*"
```

```
echo "Все аргументы (\$@): @$"
```

```
echo "Перебираем аргументы по одному:"
```

```
count=1
```

```
for arg in "$@"
```

```
do
```

```
    echo "Аргумент $count: $arg"
```

```
    ((count++))
```

```
done
```

Figure 4: Как видно из рис. 4, архив backup_self.tar.gz успешно создан. Размер файла составляет 248 байт

```
#!/bin/bash
```

```
echo "Всего аргументов: $#"
```

```
echo "Имя скрипта: $0"
```

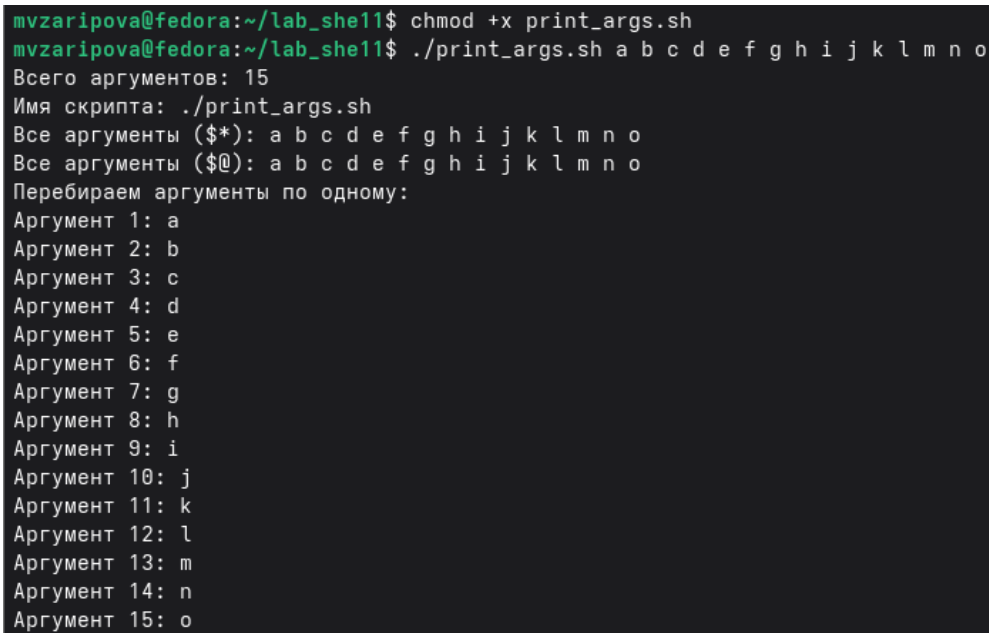
```
echo "Все аргументы (\$*): $*"
```

```
echo "Все аргументы (\$@): @$"
```

```
echo "Перебираем аргументы по одному:"
```

```
count=1
for arg in "$@"
do
    echo "Аргумент $count: $arg"
    ((count++))
done
```

Делаю скрипт исполняемым и запускаю с 15 аргументами (рис. 5).



```
mvzaripova@fedora:~/lab_she11$ chmod +x print_args.sh
mvzaripova@fedora:~/lab_she11$ ./print_args.sh a b c d e f g h i j k l m n o
Всего аргументов: 15
Имя скрипта: ./print_args.sh
Все аргументы (*): a b c d e f g h i j k l m n o
Все аргументы (@): a b c d e f g h i j k l m n o
Перебираем аргументы по одному:
Аргумент 1: a
Аргумент 2: b
Аргумент 3: c
Аргумент 4: d
Аргумент 5: e
Аргумент 6: f
Аргумент 7: g
Аргумент 8: h
Аргумент 9: i
Аргумент 10: j
Аргумент 11: k
Аргумент 12: l
Аргумент 13: m
Аргумент 14: n
Аргумент 15: o
```

Figure 5: Запуск `print_args.sh` с аргументами `a b c d e f g h i j k l m n o`

Листинг командного файла представлен на (рис. 6).

```
mvzaripova@fedora:~  
GNU nano 8.5  
#!/bin/bash  
TARGET_DIR="${1:-.}"  
  
if [ ! -d "$TARGET_DIR" ]; then  
    echo "Ошибка: $TARGET_DIR не является каталогом"  
    exit 1  
fi  
  
echo "Содержимое каталога: $TARGET_DIR"  
echo "=====  
  
for file in "$TARGET_DIR"/*; do  
    if [ -e "$file" ]; then  
        # Права доступа  
        perms=""  
        [ -r "$file" ] && perms="${perms}r" || perms="${perms}-"  
        [ -w "$file" ] && perms="${perms}w" || perms="${perms}-"  
        [ -x "$file" ] && perms="${perms}x" || perms="${perms}-"  
  
        # Тип  
        if [ -d "$file" ]; then  
            type="dir "  
        elif [ -f "$file" ]; then  
            type="file"  
        else  
            type="other"  
        fi  
  
        # Имя  
        name=$(basename "$file")  
        echo "$perms | $type | $name"  
    fi  
done
```

Figure 6: Код программы

```
#!/bin/bash  
TARGET_DIR="${1:-.}"  
  
if [ ! -d "$TARGET_DIR" ]; then  
    echo "Ошибка: $TARGET_DIR не является каталогом"  
    exit 1  
fi  
  
echo "Содержимое каталога: $TARGET_DIR"  
echo "=====  
  
for file in "$TARGET_DIR"/*; do  
    if [ -e "$file" ]; then  
        # Права доступа  
        perms=""  
        [ -r "$file" ] && perms="${perms}r" || perms="${perms}-"  
        [ -w "$file" ] && perms="${perms}w" || perms="${perms}-"  
        [ -x "$file" ] && perms="${perms}x" || perms="${perms}-"  
        # Тип  
        if [ -d "$file" ]; then  
            type="dir "  

```

```

elif [ -f "$file" ]; then
type="file"
else
type="other"
fi
# Имя
name=$(basename "$file")
echo "$perms / $type / $name"
fi
done

```

Делаю скрипт исполняемым и запускаю для текущего каталога (рис. 7).

```

mvzaripova@fedora:~/lab_shell$ chmod +x myls.sh
mvzaripova@fedora:~/lab_shell$ ./mysls.sh
Содержимое каталога: .
=====
rw- | file | 12.c
rw- | file | archive.tar
rwx | file | backup_self.sh
rwx | file | cprog
rw- | file | FILES.txt
rw- | file | input.txt
rwx | file | myls.sh
rw- | file | output.txt
rwx | file | print_args.sh
rwx | file | task1.sh
rwx | file | task2.sh
rwx | file | task3.sh
rwx | file | task4.sh
rwx | dir | testdir
mvzaripova@fedora:~/lab_shell$ ./mysls.sh /etc
Содержимое каталога: /etc
=====
r-x | dir | abrt
r-- | file | adjtime
r-- | file | aliases
r-x | dir | alsa
r-x | dir | alternatives
r-x | dir | anaconda
r-- | file | anacrontab
r-- | file | anthy-unicode.conf
r-- | file | asound.conf
r-- | file | at.deny
--- | dir | audit
r-x | dir | authselect
r-x | dir | avahi
r-x | dir | bash_completion.d
r-- | file | bashrc
r-- | file | bindresvport.blacklist
r-x | dir | binfmt.d
r-x | dir | bluetooth
--- | file | brlapi.key
r-x | dir | brltty
r-- | file | brltty.conf
r-x | dir | ceph

```

Figure 7: Запуск *mysls.sh* для текущего каталога

Код программы *count_by_ext.sh* (рис. 8).

```
mvzaripova@fedora:~  
GNU nano 8.5 count_by_ext  
#!/bin/bash  
if [ $# -ne 2 ]; then  
    echo "Использование: $0 <расширение> <каталог>"  
    echo "Пример: $0 .txt /home/user/docs"  
    exit 1  
fi  
  
EXT="$1"  
TARGET_DIR="$2"  
  
if [ ! -d "$TARGET_DIR" ]; then  
    echo "Ошибка: $TARGET_DIR не является каталогом"  
    exit 1  
fi  
  
COUNT=$(find "$TARGET_DIR" -maxdepth 1 -type f -name "$EXT" | wc -l)  
echo "Количество файлов с расширением $EXT в каталоге $TARGET_DIR: $COUNT"
```

Figure 8: Код программы

```
#!/bin/bash  
if [ $# -ne 2 ]; then  
    echo "Использование: $0 <расширение> <каталог>"  
    echo "Пример: $0 .txt /home/user/docs"  
    exit 1  
fi  
  
EXT="$1"  
TARGET_DIR="$2"  
  
if [ ! -d "$TARGET_DIR" ]; then  
    echo "Ошибка: $TARGET_DIR не является каталогом"  
    exit 1  
fi  
  
COUNT=$(find "$TARGET_DIR" -maxdepth 1 -type f -name "$EXT" | wc -l)echo "Количество  
файлов с расширением $EXT в каталоге $TARGET_DIR: $COUNT"
```

5 Выводы

При выполнении данной лабораторной работы я изучила основы программирования в оболочке ОС UNIX, научилась писать командные файлы с использованием:

1. автоматического резервного копирования с архивацией (tar);

2. обработки произвольного количества аргументов командной строки (в том числе более 10) с использованием `$@`, `$#` и цикла `for`;
3. создания аналога команды `ls` с выводом прав доступа (`-r`, `-w`, `-x`) и определения типа файлов (`-d`, `-f`);
4. подсчёта файлов по заданному расширению в указанной директории с использованием команды `find` и проверки существования каталога.

Освоены такие управляющие конструкции, как `for`, `if`, работа с переменными, подстановками, проверкой типов файлов и прав доступа, а также использование `basename`, `mkdir`, `chmod`, `tar`, `find`.

6 Ответы на контрольные вопросы

1. Что такое командная оболочка? Примеры?
Программа для взаимодействия пользователя с ОС через команды. Примеры: `sh`, `bash`, `csh`, `ksh`. Отличаются синтаксисом и возможностями.
2. Что такое POSIX?
Стандарт, обеспечивающий совместимость UNIX-подобных систем и переносимость программ.
3. Как определяются переменные и массивы в `bash`?
Переменные: `name=value`. Массивы: `arr=(a b c)` или `arr[0]=a`.
4. Назначение операторов `let` и `read`?
`let` — для арифметики. `read` — для ввода с клавиатуры.
5. Какие арифметические операции можно применять?
`+` `-` `*` `/` `%` `**` `+=` `-=` `*=` `/=` `%=`.
6. Что означает операция `(( ))`?
Арифметические вычисления без `let`.
7. Какие стандартные переменные известны?
`PATH`, `HOME`, `USER`, `PWD`, `PS1`, `RANDOM`, `SECONDS`.
8. Что такое метасимволы?
Символы со спецсмыслом: `*` `?` `[ ]` `>` `<` `|` `&` `;` `$` `\` и др.
9. Как экранировать метасимволы?
Обратным слешем (`\`), одинарными (`'`) или двойными (`"`) кавычками.
10. Как создавать и запускать командные файлы?

Создать файл с `#!/bin/bash`, сделать `chmod +x`, запустить `./файл.sh`.

11. Как определяются функции в `bash`?

`function имя { команды; }` или `имя() { команды; }`.

12. Как узнать, файл это или каталог?

`[ -d "$f" ]` — каталог, `[ -f "$f" ]` — обычный файл.

13. Назначение `set`, `typeset`, `unset`?

`set` — показывает переменные. `typeset` — задаёт свойства. `unset` — удаляет.

14. Как передаются параметры в командные файлы?

Через пробел при запуске. Внутри: `$1`, `$2`, ..., `${10}`, `$@`, `$#`.

15. Специальные переменные `bash`?

`$0` (имя скрипта), `$#` (число аргументов), `$@` (все аргументы), `$?` (код возврата), `$$` (PID процесса).